



**THE IMAGINATION UNIVERSITY PROGRAMME**

# **RVfpgaEL2 Lab 6**

## **Introduction to I/O**

## 1. Introduction

In Labs 6-10, you will learn how to use and expand RVfpgaEL2's Input/Output (I/O) system to enable the RISC-V processor to interact with peripheral devices. Below is an overview of the topics covered in these labs:

- **Lab 6:** Learn how to use the general-purpose input/output (GPIO) pins connected to the LEDs, switches, and pushbuttons on the Nexys Video Video Board
- **Lab 7:** Learn how to use the 7-segment displays available on the board
- **Lab 8:** Learn how to use timers
- **Lab 9:** Learn how to use interrupts to interface with external devices
- **Lab 10:** Lab 10 is not present for the Nexys Video Video Board, because that board does not have an SPI Accelerometer.

In this lab, we first describe the main features of a general-purpose I/O system and the one used in the RVfpgaEL2 System (Section 2). We then describe a simplified theoretical version of a generic GPIO controller (Section 3). Finally, we focus on the GPIO controller used in the VeeRwolf SoC: we first analyse its high-level specification and introduce fundamental exercises (Sections 4 and 5). We conclude the lab by analysing its low-level implementation, and introducing advanced exercises (Sections 6 and 7).

We use this same general structure in Labs 7-9. In the beginning sections, we describe the I/O controller's high-level specification (its main features, registers and their operation, and the memory map) and then introduce fundamental exercises for practice using the peripherals. In the advanced sections, we describe the controller's low-level implementation and provide exercises for modifying it and then writing programs that test the modification.

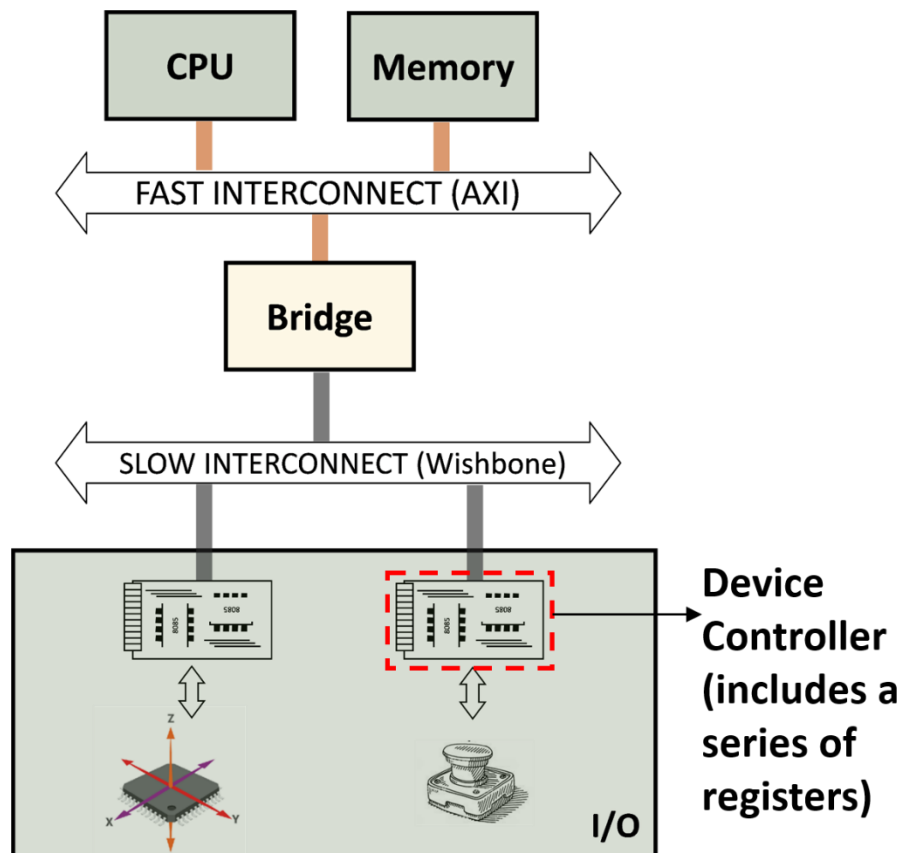
**Note to instructors:** You may choose the complexity of exercises according to your course level. For example, in a first/second year course (such as Computer Fundamentals or Computer Organization), the fundamental exercises – in this lab, Section 5 – would be suitable. However, in a more advanced course (such as Computer Architecture or Embedded System Design), both the fundamental and advanced exercises – in this lab, sections 5 to 7 – could be used.

## 2. Input/Output Architecture

Figure 1 illustrates the structure of the Von Neumann Architecture, which is composed of three main blocks: the CPU, the Memory, and the I/O System. In Labs 6-10, we focus on the CPU's interaction with input/output (I/O) devices. I/O devices are also referred to as peripherals or simply devices. We overview the role of each main unit here:

- **CPU:** the CPU is the initiator of all I/O operations. It is the *controller* (historically called “master”, but that term is deprecated) of any I/O transaction. A direct-memory-access (DMA) controller (DMAC) could also act as a controller, but it is not included in this lab.
- **Device Controller:** The *device controller* waits for read/write requests from a *controller* to perform any action. Device controllers behave as *peripherals* (formerly called “slaves,” but that term is deprecated) in the I/O system. Conceptually, a device controller consists of a series of *registers* that are accessible from the *controller*. The values of these registers instruct the *peripheral* about what action to perform.
- **The interconnect** (bus, crossbar, etc.) establishes a path between the *controller* and the *peripherals*. Interconnect is usually implemented with several layers connected

through a *bridge* that prevents certain devices from slowing down the entire system.



**Figure 1. Generic computing system**

Figure 2 shows RVfpgaEL2's I/O system. It includes the following peripherals:

- LEDs and Switches (considered a single peripheral), connected to the GPIO1 module
- 7-segment displays, connected to the System Controller module
- Timer
- UART
- Boot ROM
- SPI Flash
- SPI Accelerometer

A multiplexer selects one peripheral among the seven possibilities and connects it with the CPU. Note that a Wishbone to AXI Bridge is necessary because the peripherals use a Wishbone bus (grey colour) whereas the VeeR EL2 Core uses an AXI bridge (orange colour).

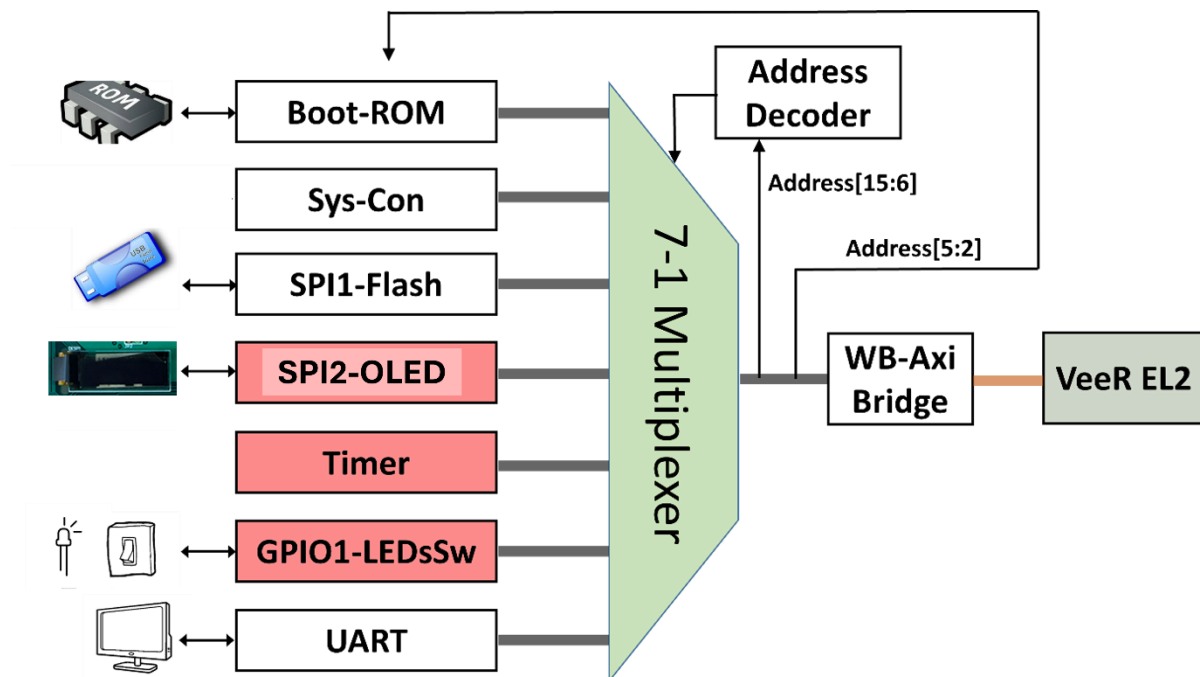


Figure 2. RVfpgaEL2's I/O system

**TASK:** Locate each of the elements of Figure 2 in the SoC. You will need to inspect the following files and directories:

[RVfpgaEL2Nexys VideoNoDDRPath]/src/VeeRwolf/veerwolf\_core.v (main file, where the elements from Figure 2 are instantiated).

[RVfpgaEL2Nexys VideoNoDDRPath]/src/VeeRwolf/Peripherals

[RVfpgaEL2Nexys VideoNoDDRPath]/src/VeeRwolf/Interconnect/WishboneInterconnect

[RVfpgaEL2Nexys VideoNoDDRPath]/src/VeeRwolf/Peripherals/SystemController/veerwolf\_syscon.v

[RVfpgaEL2Nexys VideoNoDDRPath]/src/VeeRwolf/Interconnect/WishboneInterconnect/wb\_intercon.v

[RVfpgaEL2Nexys VideoNoDDRPath]/src/VeeRwolf/Interconnect/WishboneInterconnect/wb\_intercon.vh

As described in the RVfpgaEL2 Getting Started Guide (GSG), the original VeeRwolf (<https://github.com/chipsalliance/VeeRwolf>) system-on-chip (SoC) includes only some of the peripherals shown in Figure 2: specifically, the Boot ROM, System Controller (with no 7-Segment Displays), SPI Flash and UART (shown in white in Figure 2). Remember from the GSG that VeeRwolfX SoC extends the original VeeRwolf SoC with new peripherals: a Timer, a GPIO module (shown in red in Figure 2), an SPI Accelerometer and a 7-segment display controller (that extends VeeRwolf's existing System Controller).

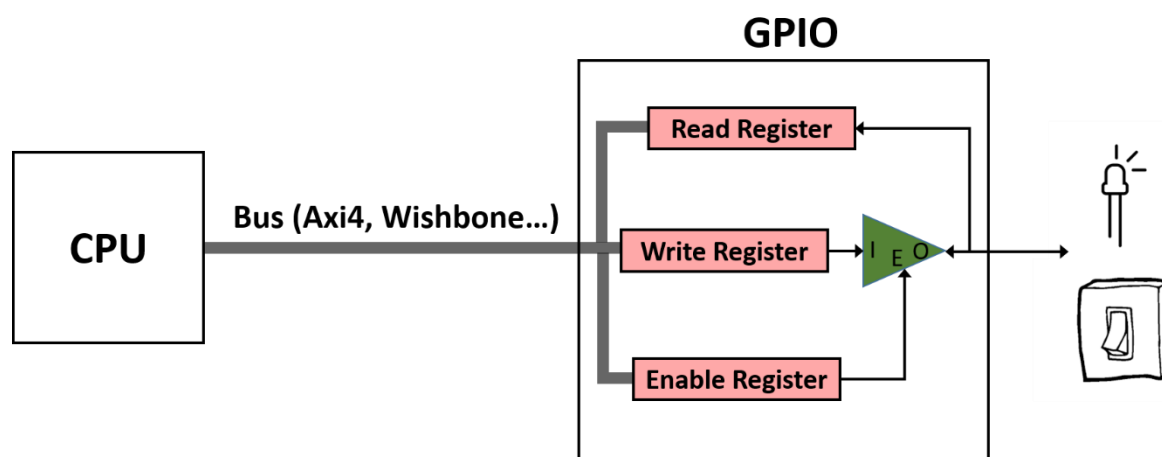
Each peripheral receives values from the processor and/or sends values back to the processor. Particular memory addresses are assigned to hold values in registers that are then used by the peripherals. These addresses are called *registers*, *memory-mapped I/O registers*, or *device controller registers*. To send a value to a peripheral, the CPU stores a value to a specified memory address (i.e., memory-mapped register). To read a value from a peripheral, the CPU loads a value from a specified memory address. Thus, a simple *load/store* operation from the CPU may configure a device, check its status, or read/write data from/onto it.

The multiplexer in Figure 2 selects the requested device controller using *Address[15:6]*. The device controllers use *Address[5:2]* to select among several registers used to control the device.

### 3. General Purpose Input/Output (GPIO)

A general-purpose I/O (GPIO) controller exposes external digital pins to the programmer. At any given time in the program, those pins can be configured as either inputs or outputs. That designation is per pin and can change throughout the program, if desired. GPIO pins can be connected to external devices such as LEDs, switches, and pushbuttons.

Figure 3 illustrates a simplified diagram for a generic GPIO module connecting one external pin to the CPU. The pin can be connected to any input/output device, such as an LED, a switch, etc. The pin is connected to a tri-state buffer, highlighted in green in the figure. This buffer allows the programmer to configure the pin as either an input or output. If the tri-state buffer is enabled, the pin acts as an output (for example, for driving an LED). If the tri-state buffer is disabled, the pin acts as an input (for example, for reading from switch values).



**Figure 3. GPIO simplified circuit**

A tri-state buffer can either act as a regular buffer (when it is enabled) or have a floating output (when it is disabled). The tri-state buffer has two inputs, E (enable) and I (input), and one output, O, and its truth table is shown in Table 1. When E is 1, the tri-state acts as a regular buffer with the output (O) and input (I) being the same. When E is 0, no connection exists between the input and output and the output (O) is not driven; O is floating. In Figure 3, to configure a pin as an output, E is 1, which allows the CPU to drive the pin. When a pin is configured as an input, E is 0, which keeps the CPU from driving the pin and allows the peripheral to drive it.

**Table 1. Tri-state truth table**

| E | I | O    |
|---|---|------|
| 0 | 0 | Hi-Z |
| 0 | 1 | Hi-Z |
| 1 | 0 | 0    |
| 1 | 1 | 1    |

The RVfpgaEL2 System uses memory-mapped I/O to read/write the values stored in these registers. For example, assume that the pin from Figure 3 is connected to a switch and that the three registers in the GPIO are mapped as follows:

- Read Register           =       Address 0x80001400
- Write Register         =       Address 0x80001404
- Enable Register        =       Address 0x80001408

To read the state of the switch, we do the following:

1. Configure the pin as an input by writing a 0 to the Enable Register (i.e., by executing a *store* of 0 to address 0x80001408).
2. Read the Read Register by executing a *load* instruction to address 0x80001400.

## 4. GPIO High-Level Specification

In this section, we first analyse the high-level specification of VeeRwolf's GPIO and then we propose one exercise that uses this peripheral.

### A. GPIO Module

The GPIO module used in VeeRwolf is from OpenCores (<https://opencores.org/projects/gpio>). The *gpio\_spec.pdf* document provided with the OpenCore's GPIO module download describes the module's high-level specification. It is available here: *[RVfpgaEL2NexysVideoNoDDRPath]/src/VeeRwolf/Peripherals/gpio/docs/gpio\_spec.pdf*. We summarize the main operation and features of the GPIO module in this lab. However, you can read the complete specifications in *gpio\_spec.pdf*.

The GPIO module has the following main features:

- It uses a Wishbone Interconnect.
- It operates as a peripheral device only.
- The user may use 1-32 GPIO pins.
- Multiple GPIO modules (also called GPIO cores) can be used in parallel to access more than 32 GPIO pins.
- All GPIO pins can be:
  - bi-directional (external bi-directional I/O cells are required in this case).
  - tri-state or open-drain enabled (external tri-state or open-drain I/O cells are required in this case).
- GPIO pins that are programmed as inputs:
  - can be registered.
  - can cause an interrupt request to the CPU.

Section 4 of the GPIO core specification describes the control and status registers available inside the GPIO module. Each of these registers is assigned to a different address as shown in Table 2. The base address for the GPIO registers is **0x80001400**.

**Table 2. GPIO registers**

| Name        | Address    | Width | Access | Description                                |
|-------------|------------|-------|--------|--------------------------------------------|
| RGPIO_IN    | 0x80001400 | 1-32  | R      | GPIO input data                            |
| RGPIO_OUT   | 0x80001404 | 1-32  | R/W    | GPIO output data                           |
| RGPIO_OE    | 0x80001408 | 1-32  | R/W    | GPIO output driver enable                  |
| RGPIO_INTE  | 0x8000140C | 1-32  | R/W    | Interrupt enable                           |
| RGPIO_PTRIG | 0x80001410 | 1-32  | R/W    | Type of event that triggers an interrupt   |
| RGPIO_AUX   | 0x80001414 | 1-32  | R/W    | Multiplex auxiliary inputs to GPIO outputs |
| RGPIO_CTRL  | 0x80001418 | 2     | R/W    | Control register                           |

|            |            |      |     |                                       |
|------------|------------|------|-----|---------------------------------------|
| RGPIO_INTS | 0x8000141C | 1-32 | R/W | Interrupt status                      |
| RGPIO_ECLK | 0x80001420 | 1-32 | R/W | Enable gpio_eclk to latch<br>RGPIO_IN |
| RGPIO_NEC  | 0x80001424 | 1-32 | R/W | Select active edge of gpio_eclk       |

Although OpenCore's GPIO module is more complex than the simplified version illustrated in Figure 3, we can still identify the three registers from Figure 3: Read (input), Write (output), and Enable. In the OpenCore's GPIO module, these registers are called, respectively: RGPIO\_IN, RGPIO\_OUT and RGPIO\_OE and are mapped to addresses 0x80001400, 0x80001404, and 0x80001408 respectively.

**TASK:** Locate the declaration of registers RGPIO\_IN, RGPIO\_OUT and RGPIO\_OE in the GPIO module, as well as the definition of their addresses. The GPIO module is here: *[RVfpgaEL2Nexys VideoNoDDRPPath]/src/VeeRwolf/Peripherals/gpio/gpio\_top.v*.

The RGPIO\_IN register latches general-purpose inputs. The RGPIO\_OUT register drives general-purpose outputs. RGPIO\_OE configures each I/O pin as an input or output. When the enable bit (within RGPIO\_OE) is set, the corresponding general-purpose output driver is enabled, and thus the pin can be connected to an output peripheral, such as an LED. When the enable bit is cleared, the output driver is operating in open-drain, also called tri-state or high impedance, mode, and thus the pin can be connected to an input peripheral, such as a switch or pushbutton.

In RVfpgaEL2-Nexys VideoA7, the lower 16 GPIO pins, pins 15:0, of the GPIO module are connected to the 16 LEDs on the Nexys Video Video board. The upper 16 GPIO pins, pins 31:16, of the GPIO controller are connected to the 16 on-board switches.

## 5. Fundamental Exercises

**Exercise 1.** Write either a RISC-V assembly program or a C program that shows a block of four lit LEDs that repeatedly moves from one side of the 16 LEDs available on the board to the other. Also include two switches that control the speed and direction. Switch[0] changes the speed and Switch[1] changes the direction as follows:

- If Switch[0] is ON (high), the lit LEDs should move quickly. Otherwise, the lit LEDs should move slowly. You may define what “quickly” and “slowly” mean, but either speed must be visible, and you must be able to detect a difference in speed just by looking at it.
- If Switch[1] is ON (high), the lit LEDs should repeatedly move from right-to-left (they start back at the right when they reach the left-most LED). Otherwise, the lit LEDs should repeatedly move from left-to-right.

**Hint:** Recall that the switches are connected to pins 31:16 of the memory-mapped I/O registers. So, to read Switch[0], you would need to write 0 to RGPIO\_OE[16] and then read the value of RGPIO\_IN[16]. You will need to configure RGPIO\_OE appropriately to access the other LEDs and switches.

Recall that you can run the same program both in RVfpgaEL2-Nexys VideoA7 and RVfpgaEL2-ViDBo. For example, this figure shows the program running on RVfpgaEL2-ViDBo:



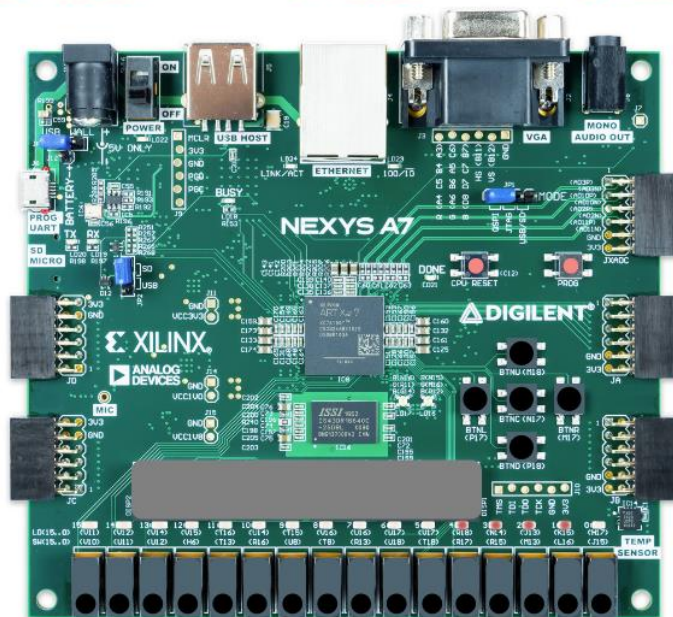
|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
|---|---|---|---|---|---|---|---|

|  |
|--|
|  |
|--|

|  |
|--|
|  |
|--|

\_\_\_\_\_

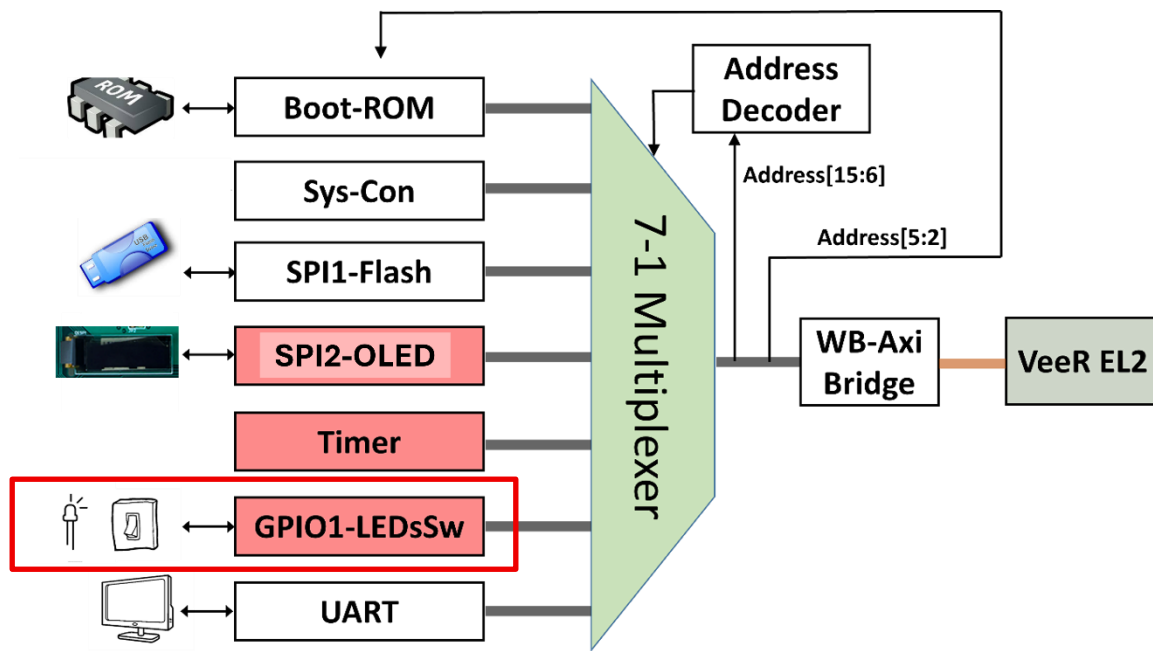
9



In this section, we describe the low-level details of the GPIO used in VeeRwolf. We then propose some exercises where you will first modify the SoC to add a new GPIO peripheral and then write a program that uses this new peripheral.

Now that you have had some experience with accessing the GPIO pins using memory-mapped I/O, let's dive into the low-level details of the GPIO. The GPIO can be divided into three main parts, as shown in Figure 4: (1) RVfpgaEL2-Nexys VideoA7' external connection to the on-board LEDs/Switches (left shaded region in Figure 4); (2) Integration of the GPIO module into the VeeRwolf SoC (middle shaded region in Figure 4); (3) Connection between the GPIO and the VeeR EL2 Core (right shaded region in Figure 4).





**Figure 4. GPIO analysis in 3 phases**

#### **i. Connection of the LEDs/Switches with the SoC**

The project's constraints file (*[RVfpgaEL2Nexys VideoNoDDRPath]/src/rvfpgaNexys Video.xdc*) defines the connection between the input/output SoC signals and the board devices. Each board device is associated with a given FPGA pin. For example, Switch[0], the right-most switch on the board, is connected through a printed circuit board (PCB) trace to FPGA pin E22.

The Nexys Video Video board includes 8 LEDs and 8 Switches. The signal that connects the 8 LEDs with the top-module of the SoC (called *rvfpgaNexys Video*, available inside file *[RVfpgaEL2Nexys VideoNoDDRPath]/src/rvfpgaNexys Video.sv*) is called *o\_led[7:0]*, and the signal that connects the 8 Switches with top-module is called *i\_sw[7:0]*. Figure 5 shows the section of the Xilinx design constraint (xdc) file, *rvfpganexys.xdc* (available in *[RVfpgaEL2Nexys VideoNoDDRPath]/src*) where these 32 connections between the signal and FPGA pin are defined.

```

32  ## LEDs
33  set_property -dict { PACKAGE_PIN T14  IOSTANDARD LVCMOS25 } [get_ports { o_led[0] }]; #IO_L15P_T2_DQS_13 Sch=led[0]
34  set_property -dict { PACKAGE_PIN T15  IOSTANDARD LVCMOS25 } [get_ports { o_led[1] }]; #IO_L15N_T2_DQS_13 Sch=led[1]
35  set_property -dict { PACKAGE_PIN T16  IOSTANDARD LVCMOS25 } [get_ports { o_led[2] }]; #IO_L17P_T2_13 Sch=led[2]
36  set_property -dict { PACKAGE_PIN U16  IOSTANDARD LVCMOS25 } [get_ports { o_led[3] }]; #IO_L17N_T2_13 Sch=led[3]
37  set_property -dict { PACKAGE_PIN V15  IOSTANDARD LVCMOS25 } [get_ports { o_led[4] }]; #IO_L14N_T2_SRCC_13 Sch=led[4]
38  set_property -dict { PACKAGE_PIN W16  IOSTANDARD LVCMOS25 } [get_ports { o_led[5] }]; #IO_L16N_T2_13 Sch=led[5]
39  set_property -dict { PACKAGE_PIN W15  IOSTANDARD LVCMOS25 } [get_ports { o_led[6] }]; #IO_L16P_T2_13 Sch=led[6]
40  set_property -dict { PACKAGE_PIN Y13  IOSTANDARD LVCMOS25 } [get_ports { o_led[7] }]; #IO_L5P_T0_13 Sch=led[7]
41
42
43  ## Buttons
44  #set_property -dict { PACKAGE_PIN B22  IOSTANDARD LVCMOS12 } [get_ports { btnc }]; #IO_L20N_T3_16 Sch=btnc
45  #set_property -dict { PACKAGE_PIN D22  IOSTANDARD LVCMOS12 } [get_ports { btnd }]; #IO_L22N_T3_16 Sch=btnd
46  #set_property -dict { PACKAGE_PIN C22  IOSTANDARD LVCMOS12 } [get_ports { btnl }]; #IO_L20P_T3_16 Sch=btnl
47  #set_property -dict { PACKAGE_PIN D14  IOSTANDARD LVCMOS12 } [get_ports { btnr }]; #IO_L6P_T0_16 Sch=btnr
48  #set_property -dict { PACKAGE_PIN F15  IOSTANDARD LVCMOS12 } [get_ports { btneu }]; #IO_0_16 Sch=btneu
49  #set_property -dict { PACKAGE_PIN G4  IOSTANDARD LVCMOS15 } [get_ports { cpu_resetrn }]; #IO_L12N_T1_MRCC_35 Sch=cpu_resetrn
50
51
52  ## Switches
53  set_property -dict { PACKAGE_PIN E22  IOSTANDARD LVCMOS12 } [get_ports { i_sw[0] }]; #IO_L22P_T3_16 Sch=sw[0]
54  set_property -dict { PACKAGE_PIN F21  IOSTANDARD LVCMOS12 } [get_ports { i_sw[1] }]; #IO_25_16 Sch=sw[1]
55  set_property -dict { PACKAGE_PIN G21  IOSTANDARD LVCMOS12 } [get_ports { i_sw[2] }]; #IO_L24P_T3_16 Sch=sw[2]
56  set_property -dict { PACKAGE_PIN G22  IOSTANDARD LVCMOS12 } [get_ports { i_sw[3] }]; #IO_L24N_T3_16 Sch=sw[3]
57  set_property -dict { PACKAGE_PIN H17  IOSTANDARD LVCMOS12 } [get_ports { i_sw[4] }]; #IO_L6P_T0_15 Sch=sw[4]
58  set_property -dict { PACKAGE_PIN J16  IOSTANDARD LVCMOS12 } [get_ports { i_sw[5] }]; #IO_0_15 Sch=sw[5]
59  set_property -dict { PACKAGE_PIN K13  IOSTANDARD LVCMOS12 } [get_ports { i_sw[6] }]; #IO_L19P_T3_A22_15 Sch=sw[6]
60  set_property -dict { PACKAGE_PIN M17  IOSTANDARD LVCMOS12 } [get_ports { i_sw[7] }]; #IO_25_15 Sch=sw[7]

```

**Figure 5. Connection of `i_sw[7:0]` with the on-board switches and `o_led[7:0]` with the on-board LEDs (file `rvfpganexys.xdc`).**

The top-module (`rvfpganexys`) shows these two signals connected to the SoC (top of Figure 6), and the end of that module shows their connection with the `veerwolf_core` module (bottom of Figure 6). Note that the `i_sw` and `o_led` signals are merged in signal `io_data` (line 256), a 32-bit input/output signal connected with the GPIO in the `veerwolf_core` module (as will be shown later, in Figure 7). Moreover, note that the `o_led` signal is latched through an intermediate signal, `gpio_out` (line 265).

```

`default_nettype none
module rvfpganexys
  #(parameter bootrom_file = "boot_main.mem")
  (input wire      clk,
   input wire      rstn,
   output wire [12:0] ddram_a,
   output wire [2:0] ddram_ba,
   output wire      ddram_ras_n,
   output wire      ddram_cas_n,
   output wire      ddram_we_n,
   output wire      ddram_cs_n,
   output wire [1:0] ddram_dm,
   inout wire [15:0] ddram_dq,
   inout wire [1:0] ddram_dqs_p,
   inout wire [1:0] ddram_dqs_n,
   output wire      ddram_clk_p,
   output wire      ddram_clk_n,
   output wire      ddram_cke,
   output wire      ddram_odt,
   output wire      o_flash_cs_n,
   output wire      o_flash_mosi,
   input wire      i_flash_miso,
   input wire      i_uart_rx,
   output wire      o_uart_tx,
   inout wire [15:0] i_sw,
   output reg [15:0] o_led,
   output reg [7:0]  AN,
   output reg        CA, CB, CC, CD, CE, CF, CG,

```

```
.io_data      ({i_sw[15:0],gpio_out[15:0]}),
.MV (MV),
.Digits_Bits ({CA,CB,CC,CD,CE,CF,CG}),
.o_accel_sclk (accel_sclk),
.o_accel_cs_n (o_accel_cs_n),
.o_accel_mosi (o_accel_mosi),
.i_accel_miso (i_accel_miso));

always @(posedge clk_core) begin
    o_led[15:0] <= gpio_out[15:0];
end
```

**Figure 6. Connection of the LEDs and the switches with the top-level module (rvfpganexys.sv)**

**TASKS:** Follow these two signals (*i\_sw* and *o\_led*) from the constraints file to the VeeRwolf SoC module (where they are merged in *io\_data*). You will need to inspect the following files:

[RVfpgaEL2Nexys VideoNoDDRPath]/src/rvfpgaNexys Video.xdc  
 [RVfpgaEL2Nexys VideoNoDDRPath]/src/rvfpgaNexys Video.sv  
 [RVfpgaEL2Nexys VideoNoDDRPath]/src/VeeRwolf/veerwolf\_core.v

In the previous section we said that in RVfpgaEL2-Nexys VideoA7 the 16 first GPIO pins (15 to 0) of the GPIO module are connected to the 16 on-board LEDs, whereas the 16 last GPIO pins (31 to 16) of the GPIO controller are connected with the 16 on-board switches. Does this correspond with the implementation described in this section and in Figure 7?

## ii. Integration of the GPIO module in the SoC

In the **veerwolf\_core** module ([RVfpgaEL2Nexys VideoNoDDRPath]/src/VeeRwolf/veerwolf\_core.v), the GPIO module is instantiated and integrated into the SoC (see Figure 7).

```
// GPIO - Leds and Switches
wire [31:0] en_gpio;
wire      gpio_irq;

`ifdef ViDBo
`elsif Pipeline
`else

wire [31:0] i_gpio;
wire [31:0] o_gpio;

bidirec gpio0 (.oe(en_gpio[0]), .inp(o_gpio[0]), .outp(i_gpio[0]), .bidir(io_data[0]));
bidirec gpio1 (.oe(en_gpio[1]), .inp(o_gpio[1]), .outp(i_gpio[1]), .bidir(io_data[1]));
bidirec gpio2 (.oe(en_gpio[2]), .inp(o_gpio[2]), .outp(i_gpio[2]), .bidir(io_data[2]));
bidirec gpio3 (.oe(en_gpio[3]), .inp(o_gpio[3]), .outp(i_gpio[3]), .bidir(io_data[3]));
bidirec gpio4 (.oe(en_gpio[4]), .inp(o_gpio[4]), .outp(i_gpio[4]), .bidir(io_data[4]));
bidirec gpio5 (.oe(en_gpio[5]), .inp(o_gpio[5]), .outp(i_gpio[5]), .bidir(io_data[5]));
bidirec gpio6 (.oe(en_gpio[6]), .inp(o_gpio[6]), .outp(i_gpio[6]), .bidir(io_data[6]));
bidirec gpio7 (.oe(en_gpio[7]), .inp(o_gpio[7]), .outp(i_gpio[7]), .bidir(io_data[7]));
bidirec gpio8 (.oe(en_gpio[8]), .inp(o_gpio[8]), .outp(i_gpio[8]), .bidir(io_data[8]));
bidirec gpio9 (.oe(en_gpio[9]), .inp(o_gpio[9]), .outp(i_gpio[9]), .bidir(io_data[9]));
bidirec gpio10 (.oe(en_gpio[10]), .inp(o_gpio[10]), .outp(i_gpio[10]), .bidir(io_data[10]));
bidirec gpio11 (.oe(en_gpio[11]), .inp(o_gpio[11]), .outp(i_gpio[11]), .bidir(io_data[11]));
bidirec gpio12 (.oe(en_gpio[12]), .inp(o_gpio[12]), .outp(i_gpio[12]), .bidir(io_data[12]));
bidirec gpio13 (.oe(en_gpio[13]), .inp(o_gpio[13]), .outp(i_gpio[13]), .bidir(io_data[13]));
bidirec gpio14 (.oe(en_gpio[14]), .inp(o_gpio[14]), .outp(i_gpio[14]), .bidir(io_data[14]));
bidirec gpio15 (.oe(en_gpio[15]), .inp(o_gpio[15]), .outp(i_gpio[15]), .bidir(io_data[15]));
bidirec gpio16 (.oe(en_gpio[16]), .inp(o_gpio[16]), .outp(i_gpio[16]), .bidir(io_data[16]));
bidirec gpio17 (.oe(en_gpio[17]), .inp(o_gpio[17]), .outp(i_gpio[17]), .bidir(io_data[17]));
bidirec gpio18 (.oe(en_gpio[18]), .inp(o_gpio[18]), .outp(i_gpio[18]), .bidir(io_data[18]));
bidirec gpio19 (.oe(en_gpio[19]), .inp(o_gpio[19]), .outp(i_gpio[19]), .bidir(io_data[19]));
bidirec gpio20 (.oe(en_gpio[20]), .inp(o_gpio[20]), .outp(i_gpio[20]), .bidir(io_data[20]));
bidirec gpio21 (.oe(en_gpio[21]), .inp(o_gpio[21]), .outp(i_gpio[21]), .bidir(io_data[21]));
bidirec gpio22 (.oe(en_gpio[22]), .inp(o_gpio[22]), .outp(i_gpio[22]), .bidir(io_data[22]));
bidirec gpio23 (.oe(en_gpio[23]), .inp(o_gpio[23]), .outp(i_gpio[23]), .bidir(io_data[23]));
bidirec gpio24 (.oe(en_gpio[24]), .inp(o_gpio[24]), .outp(i_gpio[24]), .bidir(io_data[24]));
bidirec gpio25 (.oe(en_gpio[25]), .inp(o_gpio[25]), .outp(i_gpio[25]), .bidir(io_data[25]));
bidirec gpio26 (.oe(en_gpio[26]), .inp(o_gpio[26]), .outp(i_gpio[26]), .bidir(io_data[26]));
bidirec gpio27 (.oe(en_gpio[27]), .inp(o_gpio[27]), .outp(i_gpio[27]), .bidir(io_data[27]));
bidirec gpio28 (.oe(en_gpio[28]), .inp(o_gpio[28]), .outp(i_gpio[28]), .bidir(io_data[28]));
bidirec gpio29 (.oe(en_gpio[29]), .inp(o_gpio[29]), .outp(i_gpio[29]), .bidir(io_data[29]));
bidirec gpio30 (.oe(en_gpio[30]), .inp(o_gpio[30]), .outp(i_gpio[30]), .bidir(io_data[30]));
bidirec gpio31 (.oe(en_gpio[31]), .inp(o_gpio[31]), .outp(i_gpio[31]), .bidir(io_data[31]));

`endif

gpio_top gpio_module(
    .wb_clk_i      (clk),
    .wb_rst_i      (wb_rst),
    .wb_cyc_i      (wb_m2s_gpio_cyc),
    .wb_adr_i      ({2'b0,wb_m2s_gpio_adr[5:2],2'b0}),
    .wb_dat_i      (wb_m2s_gpio_dat),
    .wb_sel_i      (4'b1111),
    .wb_we_i       (wb_m2s_gpio_we),
    .wb_stb_i      (wb_m2s_gpio_stb),
    .wb_dat_o      (wb_s2m_gpio_dat),
    .wb_ack_o      (wb_s2m_gpio_ack),
    .wb_err_o      (wb_s2m_gpio_err),
    .wb_inta_o     (gpio_irq),
    // External GPIO Interface

    `ifdef ViDBo
    .ext_pad_i     (i_data[31:0]),
    .ext_pad_o     (o_data[31:0]),
    `elsif Pipeline
    .ext_pad_i     (i_data[31:0]),
    .ext_pad_o     (o_data[31:0]),
    `else
    .ext_pad_i     (i_gpio[31:0]),
    .ext_pad_o     (o_gpio[31:0]),
    `endif

    .ext_padoe_o   (en_gpio));
```

**Figure 7. Integration of the GPIO module (file `veerwolf_core.v`).**

The interface of the module can be divided into two blocks: Wishbone signals (Table 3), which allow the VeeR EL2 Core to communicate with the GPIO using a controller/peripheral model, and external I/O signals (Table 4).

**Table 3. Wishbone signals**

| Port      | Width | Direction | Description                                                                 |
|-----------|-------|-----------|-----------------------------------------------------------------------------|
| wb_cyc_i  | 1     | Inputs    | Indicates valid bus cycle (core select)                                     |
| wb_adr_i  | 15    | Inputs    | Address inputs                                                              |
| wb_dat_i  | 32    | Inputs    | Data inputs                                                                 |
| wb_dat_o  | 32    | Outputs   | Data outputs                                                                |
| wb_sel_i  | 4     | Inputs    | Indicates valid bytes on data bus (during valid cycle it must be 0xf)       |
| wb_ack_o  | 1     | Output    | Acknowledgment output (indicates normal transaction termination)            |
| wb_err_o  | 1     | Output    | Error acknowledgment output (indicates an abnormal transaction termination) |
| wb_rty_o  | 1     | Output    | Not used                                                                    |
| wb_we_i   | 1     | Input     | Write transaction when asserted high                                        |
| wb_stb_i  | 1     | Input     | Indicates valid data transfer cycle                                         |
| wb_inta_o | 1     | Output    | Interrupt output                                                            |

**Table 4. External I/O signals**

| Port        | Width | Direction | Description                                                         |
|-------------|-------|-----------|---------------------------------------------------------------------|
| ext_pad_i   | 1-32  | Inputs    | GPIO inputs                                                         |
| ext_pad_o   | 1-32  | Outputs   | GPIO outputs                                                        |
| ext_padoe_o | 1-32  | Outputs   | GPIO output drivers enables (for three-state or open-drain drivers) |

As shown in Figure 7, bits 5:2 of the address provided by the core in the Wishbone bus signal `wb_m2s_gpio_adr[5:2]` are used for selecting one among the 10 available memory-mapped registers. These four bits are provided to the GPIO Core through the `wb_adr_i` signal (also shown in Figure 7).

Input `ext_pad_i` connects directly with the GPIO Read Register (RGPIO\_IN). Similarly, output `ext_pad_o` connects directly with the GPIO Write Register (RGPIO\_OUT). These two signals are connected to the board LEDs and Switches (`i_gpio`, `o_gpio`, `io_data`) through 32 tri-state buffer modules. That way, all 32 pins can be configured as inputs or outputs (determined by signal `ext_padoe_o`). In our case, the lower 16 pins, pins 15:0, are connected to the LEDs (Figure 6) and thus they must be configured as outputs; the upper 16 pins, 31:16, are connected to the switches (Figure 6) and thus they must be configured as inputs. We implement these 32 tristate buffers by including the following module at the end of the `veerwolf_core` module:

```
module bidirec (input wire oe, input wire inp, output wire outp, inout wire bidir);
    assign bidir = oe ? inp : 1'bZ ;
    assign outp = bidir;
endmodule
```

Note that the simulators do not use the tristate buffers.

**TASKS:** The board GPIO pins (`io_data`) are connected to the GPIO module through tri-state buffers (see Figure 7). Analyse the tri-state buffer for the two possible states of the enable signal (`oe=0` and `oe=1`).

Taking into account the connection between the GPIO module and the on-board LEDs/Switches, what values should the programmer assign to `en_gpio`, which, as can be seen in Figure 7, is connected with `ext_padoe_o`?

### iii. Connection between the GPIO and the VeeR EL2 Core

As shown in Figure 2, the device controllers are connected to the VeeR EL2 Core through a multiplexer and a bridge. The multiplexer selects one among the  $N$  possible peripherals (in our case,  $N = 7$ ), depending on the address generated by the CPU. The bridge translates the Wishbone signals used by the device controllers to the AXI4 signals used by the VeeR Core and vice versa (implemented in file `[RVfpgaEL2Nexys VideoNoDDRRPath]/src/VeeRwolf/Interconnect/AxiToWb/axi2wb.v`).

The 7:1 multiplexer (Figure 8) is instantiated in file `[RVfpgaEL2Nexys VideoNoDDRRPath]/src/VeeRwolf/Interconnect/WishboneInterconnect/wb_intercon.v`. Then, the `wb_intercon` module is instantiated in file `[RVfpgaEL2Nexys VideoNoDDRRPath]/src/VeeRwolf/Interconnect/WishboneInterconnect/wb_intercon.vh`. This latter file is included in the `veerwolf_core` module located here: `[RVfpgaEL2Nexys VideoNoDDRRPath]/src/VeeRwolf/veerwolf_core.v`.

```

104 wb_mux
105   #(.num_slaves (7),
106     .MATCH_ADDR ({32'h00000000, 32'h00001000, 32'h00001040, 32'h00001100, 32'h00001200, 32'h00001400, 32'h00002000}),
107     .MATCH_MASK ({32'hffffff00, 32'hffffffc0, 32'hffffffc0, 32'hffffffc0, 32'hffffffc0, 32'hffffffc0, 32'hffffff00}))
108   wb_mux_io
109     (.wb_clk_i (wb_clk_i),
110      .wb_rst_i (wb_rst_i),
111      .wbm_adr_i (wb_io_adr_i),
112      .wbm_dat_i (wb_io_dat_i),
113      .wbm_sel_i (wb_io_sel_i),
114      .wbm_we_i (wb_io_we_i),
115      .wbm_cyc_i (wb_io_cyc_i),
116      .wbm_stb_i (wb_io_stb_i),
117      .wbm_cti_i (wb_io_cti_i),
118      .wbm_bte_i (wb_io_bte_i),
119      .wbm_dat_o (wb_io_dat_o),
120      .wbm_ack_o (wb_io_ack_o),
121      .wbm_err_o (wb_io_err_o),
122      .wbm_rty_o (wb_io_rty_o),
123      .wbs_adr_o ({wb_rom_adr_o, wb_sys_adr_o, wb_spi_flash_adr_o, wb_spi_oled_adr_o, wb_ptc_adr_o, wb_gpio_adr_o, wb_uart_adr_o}),
124      .wbs_dat_o ({wb_rom_dat_o, wb_sys_dat_o, wb_spi_flash_dat_o, wb_spi_oled_dat_o, wb_ptc_dat_o, wb_gpio_dat_o, wb_uart_dat_o}),
125      .wbs_sel_o ({wb_rom_sel_o, wb_sys_sel_o, wb_spi_flash_sel_o, wb_spi_oled_sel_o, wb_ptc_sel_o, wb_gpio_sel_o, wb_uart_sel_o}),
126      .wbs_we_o ({wb_rom_we_o, wb_sys_we_o, wb_spi_flash_we_o, wb_spi_oled_we_o, wb_ptc_we_o, wb_gpio_we_o, wb_uart_we_o}),
127      .wbs_cyc_o ({wb_rom_cyc_o, wb_sys_cyc_o, wb_spi_flash_cyc_o, wb_spi_oled_cyc_o, wb_ptc_cyc_o, wb_gpio_cyc_o, wb_uart_cyc_o}),
128      .wbs_stb_o ({wb_rom_stb_o, wb_sys_stb_o, wb_spi_flash_stb_o, wb_spi_oled_stb_o, wb_ptc_stb_o, wb_gpio_stb_o, wb_uart_stb_o}),
129      .wbs_cti_o ({wb_rom_cti_o, wb_sys_cti_o, wb_spi_flash_cti_o, wb_spi_oled_cti_o, wb_ptc_cti_o, wb_gpio_cti_o, wb_uart_cti_o}),
130      .wbs_bte_o ({wb_rom_bte_o, wb_sys_bte_o, wb_spi_flash_bte_o, wb_spi_oled_bte_o, wb_ptc_bte_o, wb_gpio_bte_o, wb_uart_bte_o}),
131      .wbs_dat_i ({wb_rom_dat_i, wb_sys_dat_i, wb_spi_flash_dat_i, wb_spi_oled_dat_i, wb_ptc_dat_i, wb_gpio_dat_i, wb_uart_dat_i}),
132      .wbs_ack_i ({wb_rom_ack_i, wb_sys_ack_i, wb_spi_flash_ack_i, wb_spi_oled_ack_i, wb_ptc_ack_i, wb_gpio_ack_i, wb_uart_ack_i}),
133      .wbs_err_i ({wb_rom_err_i, wb_sys_err_i, wb_spi_flash_err_i, wb_spi_oled_err_i, wb_ptc_err_i, wb_gpio_err_i, wb_uart_err_i}),
134      .wbs_rty_i ({wb_rom_rty_i, wb_sys_rty_i, wb_spi_flash_rty_i, wb_spi_oled_rty_i, wb_ptc_rty_i, wb_gpio_rty_i, wb_uart_rty_i}));

```

**Figure 8. 7:1 multiplexer selects the peripheral to connect to the CPU (`wb_intercon.v`).**

The multiplexer selects which peripheral to read or write, connecting the CPU (`wb_io_*` signals) with the Wishbone Bus of one peripheral, depending on the address. For example, if the address generated by the CPU is in the range 0x80001400-0x8000143F, the GPIO peripheral is selected, and thus signals `wb_io_*` will be connected with signals `wb_gpio_*`.

The multiplexer is implemented in file `[RVfpgaEL2Nexys VideoNoDDRRPath]/src/VeeRwolf/Interconnect/WishboneInterconnect/wb_intercon_1.2.2-r1/rtl/verilog/wb_mux.v`.



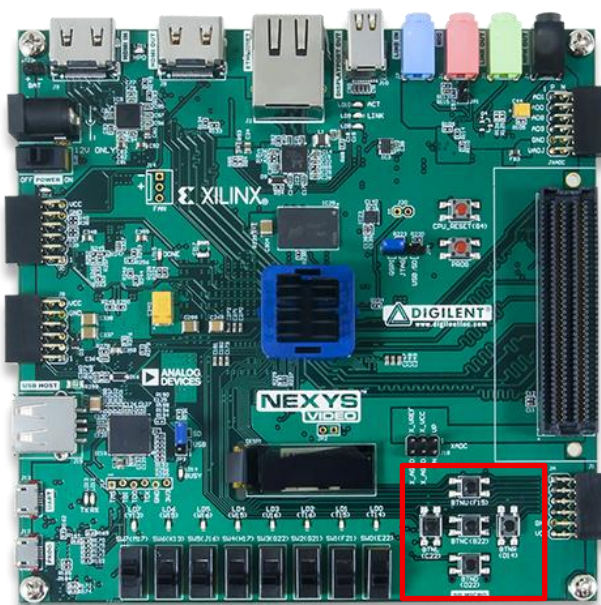
**TASK:** Analyse in detail the implementation of the multiplexer. You can focus on the GPIO-related signals (*wb\_gpio\_\**). You will need to inspect the following files:

[RVfpgaEL2Nexys VideoNoDDRPath]/src/VeeRwolf/Peripherals/SystemController/veerwolf\_syscon.v  
 [RVfpgaEL2Nexys VideoNoDDRPath]/src/VeeRwolf/Interconnect/WishboneInterconnect/wb\_intercon.v  
 [RVfpgaEL2Nexys VideoNoDDRPath]/src/VeeRwolf/Interconnect/WishboneInterconnect/wb\_intercon.vh  
 [RVfpgaEL2Nexys VideoNoDDRPath]/src/VeeRwolf/Interconnect/WishboneInterconnect/wb\_intercon\_1.2.2-r1/rtl/verilog/wb\_mux.v

Understanding this part of the SoC is important not only for this lab but also for future labs. The simulation performed in the next section can help you in understanding it if you extend the simulation by adding new signals related with the multiplexer.

## 7. Advanced Exercises

**Exercise 2.** Expand **RVfpgaEL2-Nexys VideoA7** and **RVfpgaEL2-ViDBo** to support the five on-board pushbuttons. The pushbuttons are shown in Figure 9. The five buttons are named according to their location: up, down, left, right, and center – BTNU, BTND, BTNL, BTNR, BTNC.



**Figure 9. Pushbuttons on Nexys Video Video FPGA Board**

- Given that the maximum size of the GPIO module that we are using (*gpio\_top*) is 32, which is the number of I/O pins that we have (16 LEDs + 16 Switches), you need to include another instance of the GPIO module in VeeRwolf, as well as 5 new tri-state buffers and all the necessary signals.
- Extend the multiplexer instantiation (explained in detail in the previous section, see Figure 8) for connecting the new peripheral with the core (files *wb\_intercon.v* and *wb\_intercon.vh*). You must create and connect the signals for the new gpio



instantiation and assign addresses to its registers. Use the addresses starting at 0x80001800, which are free, for mapping the registers exposed by the new GPIO controller (remember that the 16 most significant bits in parameter MATCH\_ADDR are set to 0).

**NOTE:** You can automate the process of extending the multiplexer with the help of a script instead of doing it by hand. For that purpose, follow the next steps:

- Download the Wishbone Interconnect Utilities from:  
[https://github.com/olofk/wb\\_intercon/tree/1250154467e4a5658043f4be3945fc15a7808551](https://github.com/olofk/wb_intercon/tree/1250154467e4a5658043f4be3945fc15a7808551)
- Unzip the downloaded file and go into the **sw** folder.
- Run the following command, with a **config.yml** file that specifies the number of peripherals and their mapping:

```
python3 wb_intercon_gen2.py config.yml
```

For example, a **config.yml** file that creates a multiplexer for the default peripherals plus the pushbuttons would look like this: (you can find it at [RVfpgaEL2Nexys VideoNoDDRRPath]\src\VeeRwolf\Interconnect\WishboneInterconnect\config.yml)

```
files_root: .
vlnv: ::wb_intercon:0
parameters:
  masters:
    io:
      slaves : [rom, sys, spi_flash, spi_oled, ptc,
gpio, uart]
  slaves:
    rom:
      offset : 0x00000000
      size : 0x00001000
    sys:
      offset : 0x00001000
      size : 0x00000040
    spi_flash:
      offset : 0x00001040
      size : 0x00000040
    spi_oled:
      offset : 0x00001100
      size : 0x00000040
    ptc:
      offset : 0x00001200
      size : 0x00000040
    gpio:
      offset : 0x00001400
      size : 0x00000040
    uart:
      offset : 0x00002000
      size : 0x00001000
```

You will obtain the two files that implement the multiplexer, **wb\_intercon.v** and **wb\_intercon.vh** which you can then use in your extended SoC.

**Exercise 3.** Write a RISC-V assembly program and a C program that displays a binary number on the LEDs that starts at the value 1 and increments by 1. Include an empty loop for waiting between displaying each incremented value so that the values are viewable by the human eye. Read BTN0 and BTN1 through the peripheral implemented in Exercise 2. Use BTN0 to change the speed of the count and BTN1 to restart the count whenever it is pressed.